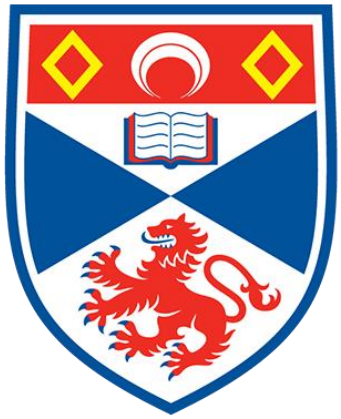
CS3099: Junior Honours Project

Visualisation of Film & TV Wikidata



University of
St Andrews

210001539 220003092 210018010 210017638 210017213

19 March 2024

Abstract

Our project is a website that visualises the relationships between films in the Wikidata database. Users can explore connections between films based on various attributes in an interactable node map. A detailed information page contains helpful widgets to provide further insights about each film.

In the development of this product, our team primarily worked on the database API. This provided an interface for each visualisation to query the necessary data, with multiple customisable and specialised endpoints designed for different components. In addition, we implemented the interesting facts and map widget.

Declaration

We declare that the material submitted for assessment is our own work except where credit is explicitly given to others by citation or acknowledgement. This work was performed during the current academic year except where otherwise stated. The main text of this project report is 7584 words long, including project specification and plan.

In submitting this project report to the University of St Andrews, we give permission for it to be made available for use in accordance with the regulations of the University Library. We also give permission for the report to be made available on the Web, for this work to be used in research within the University of St Andrews, and for any software to be released on an open-source basis.

We retain the copyright in this work, and ownership of any resulting intellectual property.

Contents

1. Introduction 1

2. Project management 2

2.1 Software Development Methodology 2

2.1.1 Scrum Model 2

2.1.2 Evaluation 2

2.1.3 Future Considerations 3

2.2 Development Tools 3

2.3 Supergroup Interaction 4

3. Project Details 5

3.1 System Functionality 5

3.1.1 Core Features 5

3.1.2 Extension Features 6

3.1.3 Software Architecture 7

3.1.4 Group Contributions 8

3.1.5 Accessibility 8

3.2 API 12

3.2.1 Technology 12

3.2.2 Design 13

3.2.3 Endpoints 14

3.3 Widgets 15

3.3.1 Interesting Facts Widget 15

3.3.2 Map Widget 17

3.4 Testing 18

4. Evaluation and Critical Appraisal 22

4.1 Supergroup 22

4.1.1 Original Objectives 22

4.1.2 Evaluation 22

4.2 Group 23

4.2.1 Original Objectives 23

4.2.2 Evaluation 23

4.3 Similar Systems 24

5. Conclusions 26

References 27

Appendices **28**

Appendix A: User Guide **28**

Appendix B: User Stories **29**

Appendix C: Interim Reports **31**

Appendix D: Scrum Reports **32**

Appendix E: API Documentation **38**

Appendix F: Supergroup Design Document **39**

Appendix G: Supergroup Meeting Minutes **41**

1. Introduction

It can be intimidating to navigate the complex history of cinema in the face of an abundance of data about many thousands of films. How does one sift through the intricate web of actors, directors, genres, ratings, and more in order to obtain insightful information?

This problem served as the motivation for our project. We wanted to create a platform which displays data for an extensive range of films in a way that captivates and informs. The resulting product is FilmVis, a data powered web application. It harnesses the immense potential of Wikidata to create dynamic visualisations that portray relationships within the cinematic landscape. From the central node map which shows how various films can be linked through common attributes, to radar charts displaying the distribution of distinct features of each film, our product offers a holistic view of data collected from the film industry.

We encourage the use of FilmVis not just for trivia enthusiasts, but for anyone wanting to dive deeper into the connections between their favourite films. We empower film lovers by providing the tools for them to explore connections between their favourite films, facilitating discovery of new cinematic territory.

Our group's primary purpose was to provide the API to interface between the films database and the visualisations. We collated a list of endpoints to fetch the necessary data from the database and serve it to the front end groups. Beyond this, we created two widgets: the map widget, which displays the filming location, and the facts widget, which algorithmically assembles a collection of interesting facts for each movie.

We take pride in our cohesive final product, which was made possible by maintaining a common back end for all parts of the project. Instead of each group working separately on their own pages with different data sources to be joined together later, we collaborated closely to ensure unity from the start. This approach allowed us to seamlessly integrate our efforts and create an efficient and complete website.

2. Project management

This part details and evaluates our software development methodology, including our management tools and the interactions between our group and the supergroup.

2.1 Software Development Methodology

Throughout the life cycle of our project, our team adopted the Scrum Agile development methodology to ensure a structured yet flexible approach to our work. This not only guided the progression of the project but also promoted collaboration, adaptability and continuous improvement within our team.

2.1.1 Scrum Model

Our Scrum implementation was both structured and adaptable. We assigned Scrum roles early on, leveraging individual strengths to enhance efficiency and ensure clear responsibilities. This allocation of a product owner and scrum master was crucial for maintaining our commitment to project goals through user stories, as seen in Appendix B. We chose Monday Work Management as our Scrum board, which helped us track progress and tasks clearly, aligning with Agile's iterative development philosophy. Our reports can be viewed in Appendix D.

Our adoption of sprint planning and weekly stand-ups provided a structure to our work. We planned for bi-weekly sprint cycles initially, which later transitioned to weekly cycles. This change highlights our willingness to adapt to better suit project needs and timelines. The sprint planning sessions laid the foundation for each sprint, setting clear objectives and tasks. Our stand-ups, often held twice a week, ensured ongoing communication, while sprint reflections offered opportunities to reflect on our work and identify areas for improvement.

2.1.2 Evaluation

One of the most significant highlights of using Scrum was the enhanced collaboration it aimed to promote within our team and with the broader supergroup. Our regular meetings aided inter-team and intra-team collaboration, crucial for the integration of our code and the cohesive development of our project. This interaction was important in overcoming miscommunications and ensuring a unified direction for the project.

However, our journey with Scrum was not without challenges. In the early stages, we overlooked the importance of reflecting on our sprint cycles. This omission led to missed opportunities for learning and improvement. For example, we initially struggled

with the integration of specific features due to unclear requirements, resulting in rework and delays. Recognising this, we began incorporating reflections post every sprint cycle, enabling us to identify and address such issues promptly.

2.1.3 Future Considerations

There are several adjustments we could have made to enhance our Scrum practice.

- **Better feature planning** would minimise time and effort spent on features that later prove misaligned with our project goals. This would involve better understanding of requirements at the planning stage.
- **Increased reflection** by incorporating in-depth sprint reviews and retrospective meetings would encourage continuous improvement.
- **Enhanced cross-team collaboration** would ensure a smoother integration of our components, though we improved significantly over the course of development.

Our application of the Scrum Agile development methodology was a learning experience that significantly contributed to our project's success. It taught us the benefits of adaptability, the importance of clear communication, and the advantages of a structured yet flexible approach to project management. As we move forward, these lessons will undoubtedly influence our future work, enabling us to tackle new challenges with confidence and a solid foundation in Agile principles.

2.2 Development Tools

Code repository

For source control and collaboration, we made use of the provided GitLab repositories. Initially, we used our team repository before each team in the supergroup combined our work at the end of semester 1. After this, we decided to work solely in branches in the supergroup repository to enable easier and faster integration between the groups.

Issue tracking

We also made use of the Issue Tracking feature in GitLab to formally mark issues for another Team to resolve. This provided reliable tracking of the issue to ensure it was not lost in a group chat or email chain, and that it was clear who should handle the issue.

Sprint tracking

To track and organise sprints we made use of Monday Work Management. This made it very easy to monitor progress and organise the tasks to be completed for each sprint. Our logs for each sprint can be found in Appendix D.

Documentation

To provide other teams with interactive documentation on the API we developed, we made use of Swagger. This was generated from YAML within our JavaScript application to make it easier for us to add new functionality whilst ensuring the documentation was up to date. The full documentation can be accessed through Appendix E.

2.3 Supergroup Interaction

We realised throughout the course of this project that communication between the teams is key to the success of the project. Towards the end of semester one, it became clear there had been miscommunications which had led to parts of the project not integrating correctly, or pieces of functionality being duplicated. This caused a last-minute rush to deliver correct functionality before the proof of concept deliverable.

Going into semester two, we were sure to implement changes into our overall supergroup workflow to ensure smoother collaboration. Although we did meet regularly in semester one, our semester two meetings were more detailed in what action points should be taken by each team, as can be seen in the meeting notes in Appendix G. It also helped that we got to know each other better, as we would be able to individually approach members of teams who were responsible for a certain section we needed to discuss. Throughout semester two, we ensured to integrate our portions of code regularly within the supergroup project to ensure reliability.

From our initial difficulties, we were able to take away many learnings and apply them to our work going forward, which we can confidently say were effective.

3. Project Details

This section provides an overview of the entire project, then goes into further detail regarding the parts our group developed, namely the database API and two widgets, including the testing process. A user guide can be found in Appendix A and user stories are listed in Appendix B.

3.1 System Functionality

Below is an overview of the features of the full application delivered by the supergroup and the architecture of the system as a whole, as well as how components were divided among the groups. Additionally, the accessibility features are highlighted here, due to these being mainly implemented by another group.

3.1.1 Core Features

With the aim of providing interesting visual information relating to films for users to explore, the supergroup created a set of core requirements in accordance with the project guidelines. These were all successfully implemented in the final product.

Wikidata data
The project enables the user to interact with film information sourced directly from Wikidata.

Node map visualisation
Relationships between films are displayed in a graph structure, showing the web of connections formed by their shared attributes, as shown in Figure 3.1. Each node is shown as a film poster, and more information can be found by hovering over edges.

Node map interaction
Users can search for the film that they are interested in viewing in the node map. They can also select the attributes they use to filter connections, allowing them to interact with the data and explore the parts they are curious about.

Information page
From the node map, users can select a film they want to learn more about to open an information page. This displays a comprehensive list of details about that specific film.

Web interface
The visualisation is presented on a user-friendly web interface with a functional and attractive aesthetic. In particular, the website meets the accessibility requirements, with options making it convenient to navigate for a variety of users.

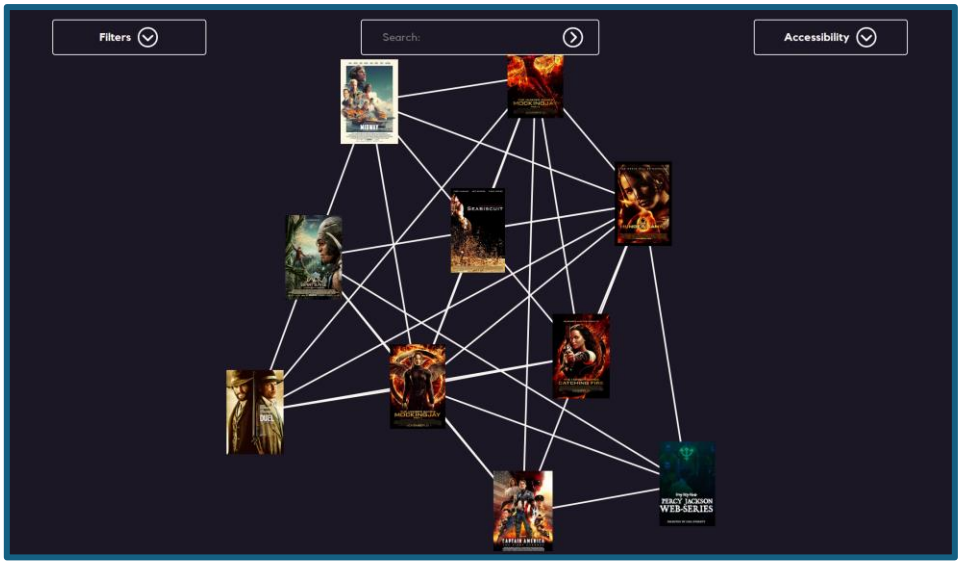


Figure 3.1. The node map front page of the website.

3.1.2 Extension Features

Beyond the core product, we implemented a number of additional widgets on the film information page as an extension, as seen on Figure 3.2.

Map widget
This is a widget that shows the filming location of the film on the world map. Each country is also highlighted based on how many films were filmed there, so users can see how their selected film compares to the rest.

Interesting facts widget
For this widget, the selected film is compared to the other films in the database and a series of interesting facts are generated based on its notable attributes. This allows users to see how each film stands out from the rest.

Timeline widget
The timeline widget allows users to view a timeline of films by the same studio, actor or director. There are various options to view and navigate the list of films, which can also be sorted by other attributes such as audience score.

Favourites widget
Using this widget, users can add a film to their favourites list for later reference. The favourites list can be cleared if desired or emailed to the user.

Ratings widget
This widget shows a radar chart displaying how much certain descriptors such as ‘funny’ and ‘romantic’ apply to the selected film. Users can contribute their own opinion by rating the film out of five stars for each descriptor.

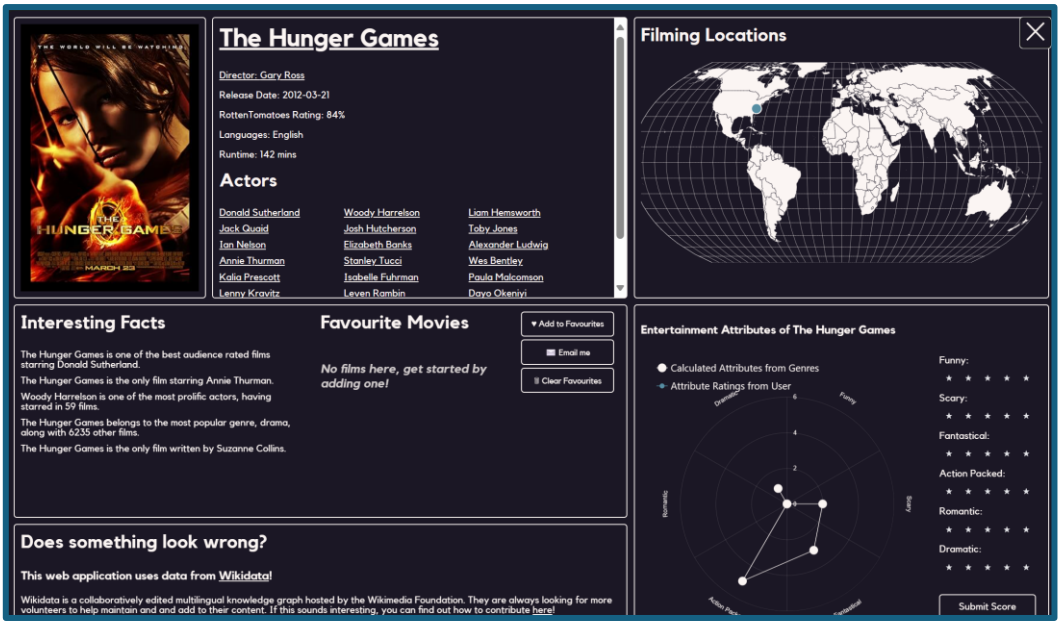


Figure 3.2. The film information page and widgets.

3.1.3 Software Architecture

The application has four distinct components which interact with each other in a mostly linear order: the React web application, the visualisations back end, the database API, and the database itself. This relationship is shown in Figure 3.3. The comprehensive film database contains the relevant processed data for our web application and is generated from the current Wikidata database. This data can then be accessed in JSON format using various complex API queries designed to meet the needs of the visualisations. When the front end receives a request from a user, the visualisations back end queries the necessary data from the database and generates the visualisations that are subsequently displayed. The web application may also send queries directly when it requires data outside of the visualisations, such as for the search functionality.

The entire system is started using the **`server.sh`** script in the repository directory. This will start up each server which is required in turn. First the API, then the node map backend and finally the front end. Each system runs as a completed separate component. We believe that this system structure works very well for this project. The clearly separate systems are easily worked on by each team, whilst maintaining strict interfaces allowed each component of the system to communicate easily according to the agreed standards. This benefit of encapsulation makes it significantly easier for a team to revise a function or restructure their code, without this affecting other teams.

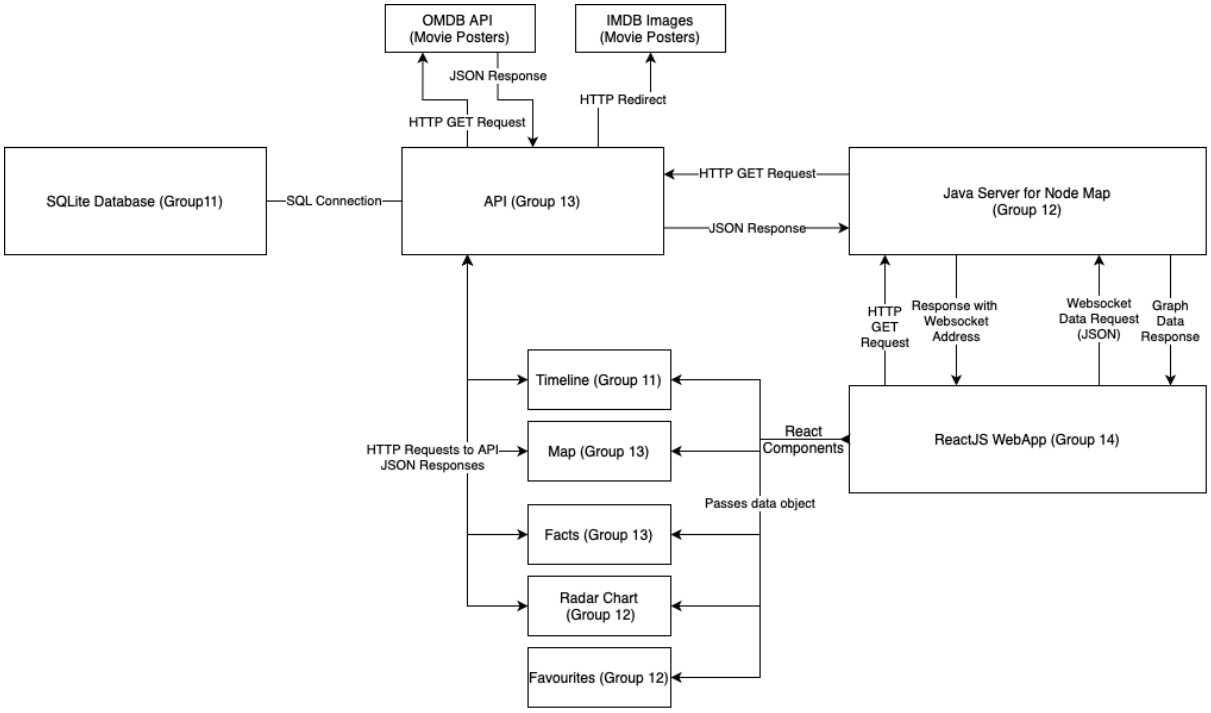


Figure 3.3. A diagram of the relationship between components of the project.

3.1.4 Group Contributions

The responsibilities for implementing the project components were divided between the four groups in the following fashion.

- **Group 11** developed the film database and the timeline widget.
- **Group 12** developed the visualisation back end, and the rating and favourite widget.
- **Group 13** developed the database API, and the map and interesting facts widget.
- **Group 14** developed the website front end.

In addition, every group contributed to our shared supergroup planning, design and testing.

3.1.5 Accessibility

Accessibility is an important matter that concerns every web application. In the UK alone, more than 1 in 5 people report a disability, and even more are temporarily or situationally impaired (Waters, 2022). Designing a website that is easy to understand and navigate not only helps create an equitable online environment for this diverse group of people but is also fundamental to overall user experience and satisfaction. On top of this, it is in many contexts a commercial and legal responsibility.

In the empathy lab with Skyscanner, we were introduced to the five categories that were the focus of the additional accessibility requirements for the project. More detailed guidelines are found in the Web Content Accessibility Guidelines (WCAG) 2.2, which is an international standard and a basis for legislation. The WCAG are founded on four key principles: perceivable, operable, understandable, and robust (W3C, 2023). As a supergroup, we have striven to apply these principles to all aspects of our product to make it usable by a wide range of people, with a focus on the categories below. Although our group did not have as many opportunities to contribute directly on the back end, we ensured our widgets followed these standards and improved clarity and organisation in our API.

Blindness

This category includes blind people, people who have sight problems or those who are focused elsewhere while using their device. These people often rely on screen readers to access website content. To aid these programs in parsing our website, we used correct HTML semantics throughout with labels and alternative text to describe visual elements. Additionally, headings and similar landmarks can be used to easily navigate the website without reading every element linearly. However, as we expected, our focus on visualisation means that this is the weakest area in terms of accessibility. Screen readers would struggle to convey all the relevant information from the node map, which is the main topic of interest. An alternative representation in plain text could have been considered.

Colour blindness

This category includes colour blind people, people under the effect of certain medication or those in a bright environment. For these users, it is essential that colour is never the only distinguishing factor for an element and that the website has high visual clarity. We implemented a light mode and a high contrast mode that can be turned on in the settings, making every element stand out as clearly as possible as visible in Figures 3.4 and 3.5. This satisfies the minimum recommended contrast levels of 3:1. Furthermore, the parts that do use colour to differentiate between elements, such as the ratings widget, always use another visual indicator such as shape as well. Unfortunately, the front end technology we chose at the start of the project does not work ideally with high zoom percentages. Nevertheless, the website caters successfully to colour blind users overall.

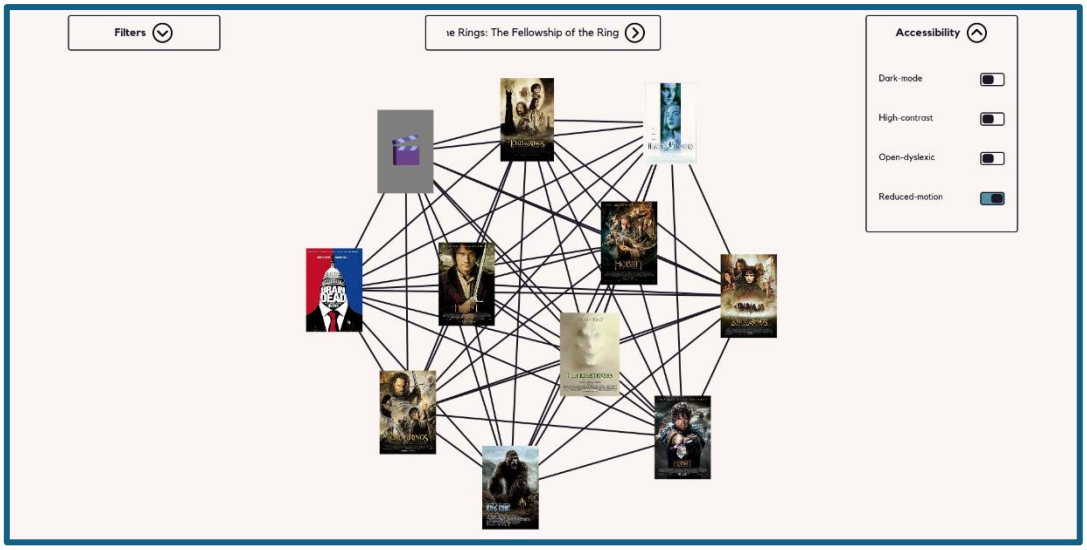


Figure 3.4. The light mode setting.

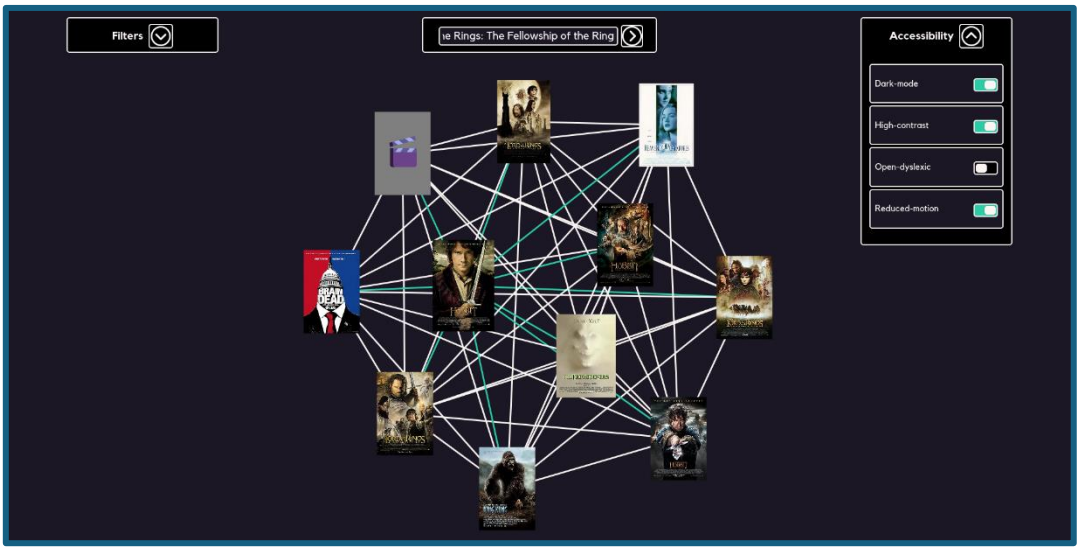


Figure 3.5. The high contrast mode setting.

Single arm

This category includes people with an amputated or broken arm, as well as those whose other hand is temporarily occupied. These users typically only use a keyboard, so it is important that every function is accessible without a mouse pointer. We implemented this by ensuring all interactable elements have a clear focus state when a user navigates to them. Moreover, most of the visual elements are also navigable using the tab key. However, more testing would have been necessary to identify blocks of content that trap keyboard users in long lists of content they wish to skip. Equally, providing keyboard users with more options to interact with the visualisations, such as by using the arrow keys, would have greatly improved their experience.

Dyslexia

This category mainly consists of individuals with dyslexia. To make our text more convenient to read for this audience, we added the option to switch the fonts to OpenDyslexic as shown in Figure 3.6. This typeface is specifically designed to minimise the problems that readers with dyslexia encounter with regular fonts (Gonzalez, 2024). This makes the website easily understandable, along with more general accomodations such as large font sizes, left aligned text, short sentences, and plenty of empty space. In general, the clear and simple design helps all users navigate the content but is in particular suitable for ones in this category.

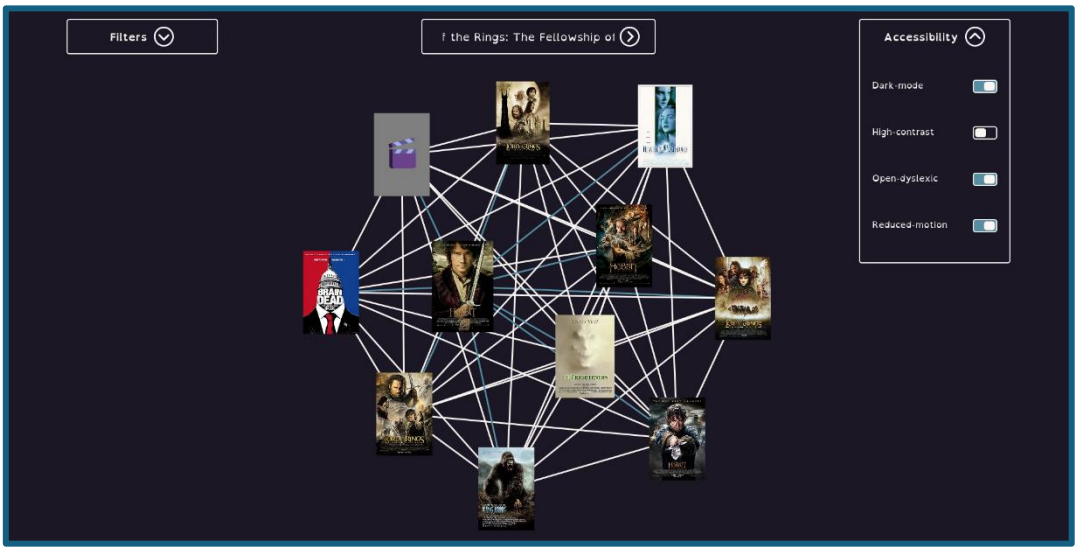


Figure 3.6. The OpenDyslexic font setting.

ADHD

This category includes people with attention deficit hyperactivity disorder, as well as other users with attention difficulties such as pregnant people. For these users, many of the same accessibility factors apply as with dyslexia. Therefore, many of the same features satisfy their needs, such as well organised and readable text. Furthermore, the reduced motion mode helps eliminate distractions. One improvement that could have been made would be to reduce the amount of content present at the same time on the information page. On the whole, though, the website is suitable at maintaining attention.

3.2 API

Our most significant contribution to the project was the database API. In this section, the overall design and technology choices are detailed together with descriptions of the individual endpoints.

3.2.1 Technology

During the planning stage, we decided on the technologies and code libraries we would use to implement the database and API. It was crucial to choose ones that not only suited the requirements, but also the experience and capabilities of the group.

JavaScript
The JavaScript language was an obvious choice for both the front and back end of a web development project. It is versatile and flexible, offering compatibility across devices and platforms as well as a many popular libraries. For the API in particular, it is well-suited for asynchronously fetching data. Furthermore, some of our group who already had experience in with this language could initially guide the rest.

Node.js

This library is widely used in web development for its efficiency and simplicity. Node.js allows JavaScript to run on the server-side, reducing the need to learn multiple languages. It is highly scalable, so it could allow us to expand concurrent connections if necessary.

Express.js
Another popular JavaScript library, Express.js provides an easy way to define routes for RESTful API requests with essential features such as error handling. This library has made the code significantly more structured and readable.

Swagger
The Swagger framework offers a standardised way to describe API endpoints with the OpenAPI Specification. This consistent design made documentation more understandable and allowed for improved collaboration. Moreover, Swagger UI automatically generates interactive documentation, reducing the effort required to maintain it manually and providing a convenient interface for testing within a browser.

JSON
The format we chose for transferring data between the database and the front end was JSON, as it is well supported by both JavaScript on our end and Java on the other group's end.

SQL

The choice of database type was one of the most important in the project and was decided as a supergroup. We considered using a NoSQL database such as MongoDB for its performance and flexible structure, aligning with the object-oriented principles of our other languages. However, the groups had less familiarity with this approach, and the lack of standardisation would make adapting more challenging. Opting for SQL was a good decision, as the relationship between data was clearly defined, ensuring consistency between groups. Additionally, SQLite was a lightweight engine that allowed us to make complex queries, despite the occasional difficulties with schema changes.

3.2.2 Design

The API has a coherent modular structure following architecture guidelines in Express.js (Moddasser, 2023). The **`app.js`** file is the entry point for the application: it sets up the middleware including JSON parsing and CORS, defines the routes, generates documentation using Swagger, and finally initialises the server. The route handlers for each endpoint are organized in separate files under the **`routes`** directory, enabling a clear separation of functionality. These files also contain JSDoc comments, which are used along with additional components and tags in **`docs`** to automatically create OpenAPI documentation in a Swagger web interface.

The connection to the database itself is established in **`db.js`** using SQLite. To maintain flexibility in case of changes in naming conventions and data structures in the front or back end, we created our own schema in **`schema.json`**. The document details the attributes and data structures in the film database, allowing us to quickly adjust to naming, type and table updates while staying consistent in the API output. This avoids additional effort spend on rewriting a multitude of queries across endpoints. Furthermore, the front end can always use the schema as an accurate reference for query parameters and results, as well as benefit from useful fields such as display name, description, and examples for each attribute. This organisational structure may have been more effective if it was implemented earlier, however, so that each group could fundamentally design around it.

To integrate with the front end in particular, Cross-Origin Resource Sharing (CORS) had to be enabled for the API. This informs the browser that content may be loaded from a different domain, which is normally disabled as a security feature.

In the design of the endpoints themselves, we followed the conventional principles of RESTful API design (Baldaniya, 2021). It was important that endpoints use consistent naming to communicate their purpose clearly. In particular, most of them use resource nouns such as **`films`** or **`attributes`** with a plural form to indicate a collection. There are exceptions where functionality such as **`search`** is not directly related to a resource, or

where dependencies made retroactive improvements impractical. The schema was greatly helpful in achieving consistency as well because it is possible to query parameter names rather than hard coding them. In addition, we used standard HTTP requests with **`GET`** indicating fetching resources, as well as status codes to precisely identify errors and invalid input. These provided a familiar environment for other groups, allowing them to immediately identify the source of issues. Above this, we included as many options as possible for each endpoint so that responses could be tailored to the situation without further processing, such as limiting the number of results or sorting them. The extensive documentation was essential in detailing all the possible parameters to other groups.

3.2.3 Endpoints

The following are the main API collections that were implemented explained in further detail. Every such group can have several individual endpoints. A full specification can be found in the documentation in Appendix E.

Films

The **`films`** resource is the most frequently queried collection by the front end. The base endpoint therefore returns a list of film objects with a significant number of options for filtering and formatting. Films may be filtered based on any attribute or attribute ID, with the ability to add any number of filters for each attribute and combine them with **`AND`** or **`OR`**. The results may also be limited to a certain number for efficiency and sorted ascending or descending by any attribute. Moreover, the matching can either be exact or accept similar words.

The results are obtained by passing a single SQLite query to the database and converting the returned object into JSON format. This complex query is constructed one attribute at a time in template literals, using the information in the schema to determine fields, joined tables and filter conditions. This is then combined with the optional parameters or their default values. Despite the large volume of data and a relational database, the queries are sufficiently fast with an adequately filtered list. Furthermore, the procedural forming of each query avoided us needing to change and retest the endpoints as development progress.

The other endpoints for this resource enable groups to search individual films more easily and to find all films linked to a given film by any of its attributes. These both reuse the same query construction with a limited set of parameters.

Attributes
Besides accessing films directly, it was necessary to be able to search individual attributes for a list of their values. The **`attributes`** collection enables querying an attribute for its values and their associated IDs with similar filtering and formatting options as films. As with that endpoint, the SQLite query is constructed from the schema and varies depending on the type of the attribute, querying a separate table for list attributes for instance.

Posters
This endpoint is responsible for loading the poster images displayed for each film on the node map. For a given film ID, the **`poster`** query locates the poster online and redirects to it. If one is not found, then the default empty poster is returned.

Search
When a user starts typing in the film search bar on the website, the front end calls the **`search`** query to efficiently return a short list of autocomplete options. The query attempts to sort results in an intuitive way, showing the ones that the user most likely intends. First, titles that begin with the search phrase are prioritised, followed by titles that include the search phrase. Between these, titles are sorted by length. This endpoint also returns the release date of each film, so that the front end can use these to differentiate between films with identical names.

Schema
The **`schema`** endpoint simply returns the entire API schema. In this document, the front end can find the names of attributes in the parameters and responses of queries, as well as additional useful display information.

3.3 Widgets

The two widgets that our group contributed to the film information page are the interesting facts list and the filming location map.

3.3.1 Interesting Facts Widget

In this widget, a list of interesting facts is displayed about the selected film. These are short sentences that relate the film and its attributes to the whole dataset, showing the user what makes each film stand out from the rest. Figure 3.7 shows an example of the widget.

The facts are drawn from the films API, in the **`stats`** routes designed specifically for this widget. The **`facts`** endpoint is the main process that queries the film details given its ID and collects a number of interesting statements related to it. For each attribute, some or all of the following statistics are checked.

- The film has the maximum value for that attribute, e.g. it has the longest duration of all films.
- The film has the minimum value for that attribute, e.g. it has the lowest possible audience rating.
- The film has a very common attribute value, e.g. it features one of the most prolific actors.
- The film has a very rare attribute value, e.g. it is one of the only films from a specific country.
- The film has one of the highest or lowest audience ratings among films with an attribute value, e.g. it is the best or worst film by a director.
- The film is the earliest or latest among films with an attribute value, e.g. it is the first or last in a genre.

To compare the film with the others in the database, two helper queries are used: **`extreme`**, which finds the relevant maximum or minimum value depending on the type of attribute, and **`counts`**, which counts the number of occurrences of each value across all films. These queries, along with the standard **`films`** query, are called asynchronously and waited on as **`Promise`** objects. The first-class functions and rest parameters in JavaScript were useful in handling these operations on different functions.

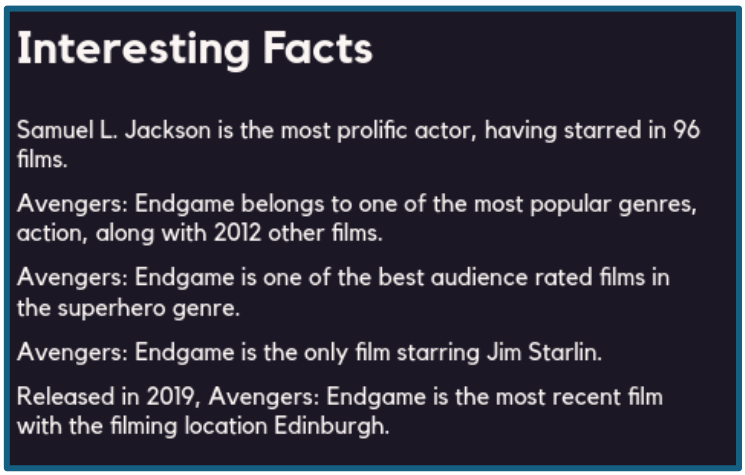


Figure 3.7. The facts widget displaying interesting facts about the film *Avengers: Endgame*.

Once these queries identify the significant values in each fact category, the given film is checked against these to find facts that are applicable. Multiple constant variables are used to determine significance, which are tuned to deliver enough facts while keeping them varied and notable. For instance, there needs to be a minimum number of films in a category for highest and lowest rated to be meaningful. Similarly, there is a maximum number of facts that can be returned from each category, otherwise a particularly high rated film might be the best in a multitude of cases. When a pertinent fact is found, the sentence is created by adding the correct names, numbers and grammatical adjustments in a template literal. A limited random selection of these is returned to be displayed in the widget.

Optimisation was crucial in the design of these queries. As the entire database must be searched multiple times for different values, adjustments were required to curtail the loading time of the film information page. This was achieved by pruning out superfluous database queries and filtering out values where the count is above a certain threshold, such as the ubiquitous English language. Ultimately, the widget is adequately efficient and runs under a few seconds at most, though further use of caching would have reduced this time to a greater extent.

3.3.2 Map Widget

To provide a more interesting visualisation of the filming location for a film, we created a map using the location names and coordinates provided by Wikidata. One or more markers are generated based on these locations and can be hovered over to reveal their names.

By making use of states in React, this widget can take data props passed in by the node map to render the map for the given film. We collaborated with members of the front end team to get the look-and-feel of the map to match the rest of the site aesthetic and integrate with the accessibility features such as dark, light and high contrast mode.



Figure 3.8. The map widget displaying the filming location North Carolina.

3.4 Testing

While creating the API, we manually tested endpoints as they were created with various parameters to ensure robustness. For this purpose, we ran the API server and used the interactive Swagger UI, which provided a testing tool for each endpoint within the browser.

This continuous testing allowed us to find any bugs and fix them before integrating the code into the main codebase, ensuring that errors in our functionality would not interfere with other teams. Testing the API directly at the endpoints rather than at the application level was an effective strategy, as the data was returned directly from our code. At the application level, data undergoes processing from other groups, which makes it harder to distinguish the origin of issues.

The widgets that we created were more difficult to test, as we were responsible for ensuring the correctness of both the front end and back end. As the data powering these widgets was served by the API endpoints, which we had already tested extensively, it was sufficient to test the information page of a handful of films on the website. This ensured the components were correctly displayed in their containers.

The table below shows an overview of the tests performed, with each endpoint being tested in normal and edge cases.

Endpoint	**Test Case**	**Type**	**Result**
`films/search`	Search with a film title in the database, fully matching	Normal	Returns the film searched for as a film object with correct details
	Search with film name in database, partially matching	Normal	Returns the first film in the database matching the partial search team as a film object with correct details
	Search with film name not in database, not matching any other film	Edge	Returns film not found error

films/links	Get links to film with valid ID	Normal	Retrieves list of films matching any of its attributes or specified with the requested formatting.
	Get links to film with invalid ID	Edge	Returns film not found error
films/filters	Get films with one filter applied	Normal	Returns a list containing all the film objects that pass the filter.
	Get films with multiple filters applied	Normal	Returns a list containing all the film objects that pass the filters.
	Get films with one invalid filter applied	Edge	Returns a maximum of 100 objects ignoring the invalid filter
	Get films with multiple filters applied, some invalid and some valid	Edge	Returns the specified number of film objects (maximum 100) containing all the film objects that pass the valid filters, ignoring the invalid filters
attributes/ filmingLocations	Get locations for film with valid ID	Normal	Returns filming locations and coordinates for a film with a valid ID
	Get locations for film with invalid ID	Edge	Does not retrieve any results

attributes	Get list of attributes without applying any filters	Normal	Returns a maximum of 100 objects of specified attribute with no filters applied
	Get list of attributes with invalid filter applied	Edge	Returns a maximum of 100 objects of specified attribute with no filters applied
poster	Get URL of poster with valid ID	Normal	Returns URL of the poster for given film
	Get URL of poster with invalid ID	Edge	Returns generic movie poster image
	Get URL of film with valid ID but no poster	Edge	Returns generic movie poster image
search	Show 5 closest matches of search term which partially matches many films	Normal	Returns 5 closest matching terms prioritising those with search term at the start
	Show 5 closest matches of search term which fully matches a film	Normal	Returns 5 closest matching terms with the fully matching result as the first
	Show 5 closest matches of search term which doesn't match any films	Edge	Returns object containing an empty results array
stats/max	Get max value of attribute	Normal	Returns maximum value of given attribute
	Get max value of invalid attribute	Edge	Returns attribute not found error

stats/counts	Get count values of attribute	Normal	Returns maximum of 100 objects containing distinct attribute values and corresponding counts
	Get count values of invalid attribute	Edge	Returns attribute not found error
stats/facts	Get facts for film with valid ID	Normal	Returns maximum of 5 facts for given film
	Get facts for film with invalid ID	Edge	Returns film not found error

4. Evaluation and Critical Appraisal

In this section, we evaluate our overall success and compare our final product to the original objectives, both for our group and the whole supergroup. As well as compare our final product to similar systems available in the public domain.

4.1 Supergroup

4.1.1 Original Objectives

At the start of the year, the initial plan for the supergroup was to produce four separate data visualisations encompassed within one website where the user could access each one individually. The four visualisations were a timeline, map, game, and the node map.

However, at the start of semester two it was decided that instead of continuing with four separate visualisations, we would focus our product principally on the node map, with data visualisations for each node being widgets within an information page for a single node. This decision was taken to ensure a more succinct final product.

Additionally, the introduction of the new accessibility requirement meant that new features had to be added to meet the requirement. As a supergroup, we decided to focus on the visually impaired and implement an accessibility menu so the user can toggle different accessibility features.

4.1.2 Evaluation

Throughout the year our supergroup has strived to keep in contact as regularly as possible to discuss plans for the future and issues that need to be resolved. This open and frequent form of communication has allowed our supergroup to set deadlines and cooperatively produce our final product.

The setup of how the tasks were split up between groups meant that a lot of communication was needed as all groups had direct dependencies with other groups. Furthermore, this meant that each group did not need to produce their own backend and front end for the product, each group instead taking responsibility for a distinct section of a cohesive product.

However, this wasn't without any challenges. At times miscommunication and lack of precise details being decided at supergroup meetings led to some problems with work being duplicated and components not integrating as intended. These problems were resolved in the second half of the year, with more emphasis on precise details being decided at meetings and regularly integrating components to make sure no major issues arose.

4.2 Group

4.2.1 Original Objectives

It was determined at the start of the year that our small group would focus on the back end of the project, together with group 11, who were responsible for retrieving the data from Wikidata and storing it in a database. The task of our group was to produce a database API with endpoints that could be utilised by the front end to produce the data visualisations. Throughout the year, our group made changes based on our experiences to improve productivity and efficiency. A main change we made was to reduce our sprint cycles to allow for more adaptability and faster turnaround when any changes or problems occurred.

Furthermore, when the supergroup was creating separate visualisations, our group planned on creating an interactive game that challenged the users to navigate from one film to another through shared attributes. As an extension to this, we planned to use the node map features to show a visual path of the journey taken by a user when playing the game. However, due to the change in our supergroup's plan to make our product more succinct by grouping all the data visualisations in one place, the original plan to make an interactive game had to be rethought. We determined that it would be more impactful to produce different mediums of data visualisation that aim to highlight interesting and notable facts relevant to a certain film. This led to the creation of our group's contribution to the information page for each film in the form of our two widgets.

4.2.2 Evaluation

At the start of the year our group assigned Scrum roles based on each member's strengths, and this worked well throughout the year leading to completion of tasks and deadlines being met successfully. This allowed for clear responsibilities to be determined within the group, leading to effective teamwork when completing tasks.

To ensure no major issues arose and progress could be monitored easily our small group had regular stand-ups twice a week to discuss relevant information and how to complete upcoming supergroup and project deadlines successfully. Our communication wasn't limited to in person meetings as we also utilised a group chat for our small group and supergroup which allowed quick resolution of small problems or issues easily. Moreover, we occasionally met to have group programming sessions. This allowed for rapid brainstorming and iteration, leading to quicker completion of code. It also increased the speed of which features could be implemented as small issues and questions could be discussed in the moment.

Our group reflected on our effectiveness during the year in our decision to switch from a two week sprint cycle to one week. This allowed us to be more adaptable as future tasks could be revaluated and assessed on a shorter time scale. Furthermore, our group showed adaptability by changing the focus of the project due to the added accessibility requirement to make the final website more succinct. This was a successful change as the end product is more cohesive than what we initially planned.

Reflecting on the overall project, we could have done a few things differently. Initially, we did not reflect enough on previous sprint cycles, which was a missed opportunity to identify mistakes and improve on them for current and future sprint cycles. For example, when approximating the time scale for some tasks, the prediction was frequently shorter than how long they took in practice. With some deeper reflection each sprint cycle, the time estimates could become slightly more accurate. For example, when a task required communicating with supergroup, more time was given to account for different groups' schedules.

When completing projects of a similar scale in the future, a main focus point will be to improve the process of planning important features. For example, we spent significant time at the start of the semester one planning an interactive game that we did not end up producing. Therefore, ensuring the initial planning stage is comprehensive and realistic is essential to not waste effort on tasks that do not align with the project goals.

4.3 Similar Systems

In the public domain, there are a lot of similar systems that offer features comparable to our website. One of most prominent film information sites is IMDb (IMDb, 1990). IMDb offers users with a wide array of information about films, including the attributes shown in FilmVis, but also allows users to rate and discuss films, read news and watch multimedia, among other features. It also has a function akin to the FilmVis node map in the 'More like this' section, which shows twelve films that relate to the main film in different ways. However, this is not the focus of the website and lacks the visual element. In the same way, our facts widget is similar to the 'Did you know' section in IMDb that shows trivia facts, connections and quotes. IMDb has a great advantage here with user-submitted facts that are unique to each film. Nonetheless, FilmVis differentiates itself with interesting statistical insights that are not immediately visible on an IMDb page. Similarly, while IMDb shows a more comprehensive view of an actor's career, the FilmVis timeline displays this in a distinctive visual manner.

Another similar application in the public domain is Rotten Tomatoes (Rotten Tomatoes, 1998). Rotten Tomatoes offers very similar features to IMDb overall, so the comparisons remain similar. However, a notable contrast is that Rotten Tomatoes

provides focuses greatly on the ratings of critics and general audiences, providing a digestible overview of opinions surrounding a film.

Lastly, an online application that is also popular among cinephiles is Letterboxd (Letterboxd, 2013), which utilises information from The Movie Database (TMDB, 2008). It places emphasis on tracking users' own film preferences, allowing them to share their reviews with their friends and catalogue their watchlists. FilmVis offers a more interesting visual interaction with the film dataset, but the two aims are very different, and the rating functionalities it offers are very limited comparted to Letterboxd.

Overall, from analysing websites that are in the public domain, providing basic information about films is very standard, along with social elements and other common functionality. Some of these are lacking in FilmVis due to the scope constraints. The greatest difference between FilmVis and the other applications, however, lies in the niche that each one focuses on. IMDb offers a very comprehensive database, Rotten Tomatoes is a hub for reviews, and Letterboxd acts as a personal film diary. In this way, FilmVis provides a unique service with its many visualisations, which was part of the motivation for its development and justifies its place alongside these larger websites.

5. Conclusions

FilmVis is a data visualisation platform that provides users with a way to navigate through a network of connected films, as well as view interesting widgets for each one. This project put most of us in unfamiliar territory. Not only did we have to organise our own roles within the group, but we also had to coordinate the role of our group within the supergroup. Overall, our supergroup handled these difficulties successfully, allocating each group a distinct section to be responsible for implementing and maintaining throughout the course of the project. As a group, we have taken a lot away from this project, we have learnt how to quickly become familiar with new technologies, whether that is creating new endpoints for an API with complex SQL queries or using React components to make widgets. As a result of our reflections, our understanding of how to effectively use Scrum has greatly increased, a skill that is directly transferable to the workplace.

A significant drawback we faced was not having a clear vision for the project in the first semester, so we spent time planning features which we later decided to abandon. This did, however, mean that we were flexible regarding the accessibility twist and therefore were able to adapt to the additional requirements together with the newly planned functionality. Another current limitation of the project is that it uses a static database of only around 15 000 films, meaning new data on Wikidata will not be reflected on our website without a manual update. An automatically updating database would be a clear area for future improvement. Relating to the parts developed by our group, we could add more interactivity to the map and increase the complexity of the API to generate more varied facts.

In the end, despite these limitations, we are proud of our collaboration and the final product we developed. The lessons learned during this project will be invaluable to all of us in the future.

References

Baldaniya, H. (2021). *Principles of Effective RESTful API Design*. Retrieved from Contentstack: https://www.contentstack.com/blog/tech-talk/effective-restful-api-design-principles

Gonzalez, A. (2024). *OpenDyslexic: A typeface for Dyslexia*. Retrieved from OpenDyslexic: https://opendyslexic.org/

IMDb. (1990). Retrieved from IMDb: https://www.imdb.com

Letterboxd. (2013). Retrieved from Letterboxd: https://letterboxd.com

Moddasser, H. (2023). *Clean Architecture in ExpressJS Applications*. Retrieved from Merlino: https://merlino.agency/blog/clean-architecture-in-express-js-applications

Rotten Tomatoes. (1998). Retrieved from Rotten Tomatoes: https://www.rottentomatoes.com

TMDB. (2008). Retrieved from TMDB: https://www.themoviedb.org

W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. Retrieved from https://www.w3.org/TR/WCAG22/

Waters, S. (2022). *Best practices for implementing great web accessibility*. Retrieved from Silktide: https://silktide.com/app/uploads/2022/03/Best-practices-for-implementing-great-web-accessibility.pdf

Appendices

Appendix A: User Guide

A formatted PDF version of the markdown **`README`** document for the project is included in the **`user-guide`** directory. This contains instructions for running the web application.

Appendix B: User Stories

Database API

- As a user, I want detailed and useful documentation of the available queries.
- As a user, I want to receive clear error messages and status codes so I can understand and resolve issues.
- As a user, I want to receive appropriately quick responses to provide a fluid experience for the end user.
- As a user, I want to retrieve a list of attributes, with the option to filter and format results, so I can obtain information about specific film attributes.
- As a user, I want to retrieve the names and coordinates of the filming locations of a given film, so they can be displayed on a map.
- As a user, I want to retrieve a list of films, with options to filter and format results, so I find connections between films to display in visualisations.
- As a user, I want to search for films by title, so I can quickly look up any film.
- As a user, I want to retrieve a list of films matching any of the attributes of a given film, with options to format results, so I can find all connections easily.
- As a user, I want to retrieve the poster of a given film to use in visualisations.
- As a user, I want to return the API schema, so I can understand the structure of the data available and create a modular program.
- As a user, I want to get the maximum value of an attribute over all films, so I can display the most extreme value for each one.
- As a user, I want to get the number of times each value of a given attribute appears over all films, so I have an overview of the data and can highlight particularly common or rare values.
- As a user, I want to get a list of fact sentences about a given film, so I can display notable statistics in the interesting facts widget.

Interesting Facts Widget

- As an end user, I want to view a list of interesting facts about the selected film, so I can learn unique and standout features of the movie compared to others in the dataset.
- As an end user, I want to learn about which films have the highest and lowest values in the dataset.

- As an end user, I want to learn about the most and least prevalent values in the dataset.
- As an end user, I want to know which films are the highest and lowest rated for a particular attribute.
- As an end user, I want to know which films are the earliest or latest released for a particular attribute.
- As an end user, I want facts to display quickly when I select a specific film.

Map Widget

- As an end user, I want to view the filming locations of a selected film on the world map, so I have an overview of where they are located.
- As an end user, I want to see the names of the filming locations as labels on the map.

As an end user, I need the map to integrate with all the accessibility features and be navigable using only a keyboard.

Appendix C: Interim Reports

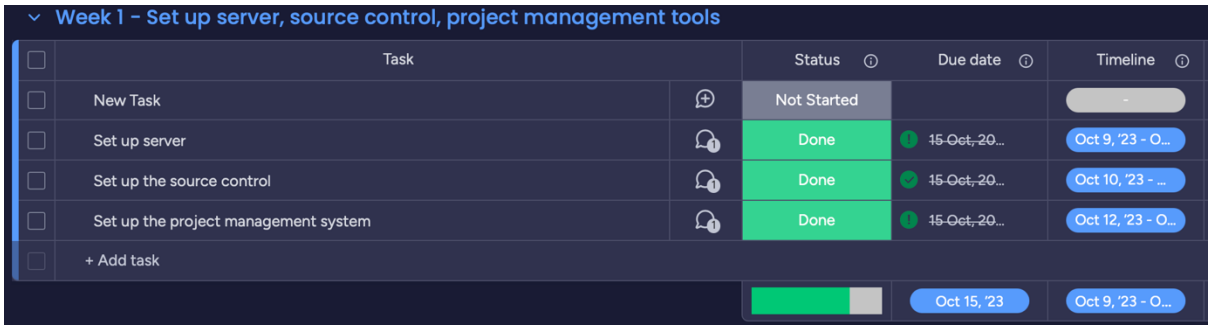
The interim reports submitted during the year are included alongside this document in the **`interim-reports`** directory.

Appendix D: Scrum Reports

Week 9th Oct

Sprint Goal: Set up server, source control, project management tools.

Impediments: All tasks for this week were implemented smoothly and efficiently with no problems encountered.

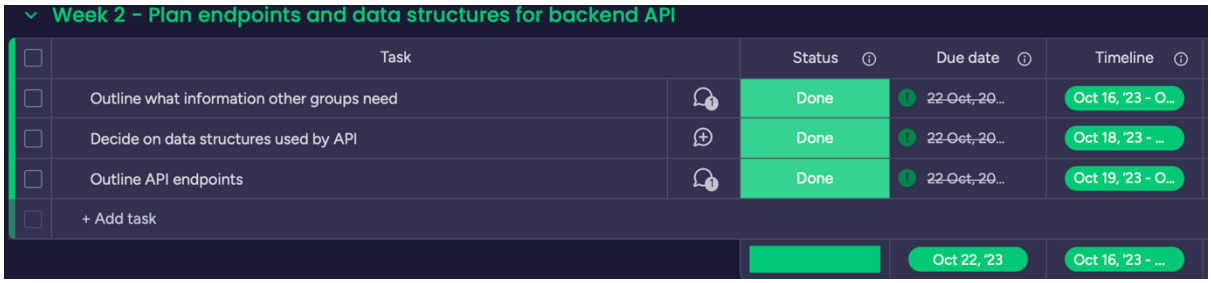


Upcoming Tasks: Plan endpoints and data structures for backend API.

Week 16th Oct

Sprint Goal: Plan endpoints and data structures for backend API.

Impediments: We found it challenging to future-proof the endpoints to ensure that all required ones are provided.

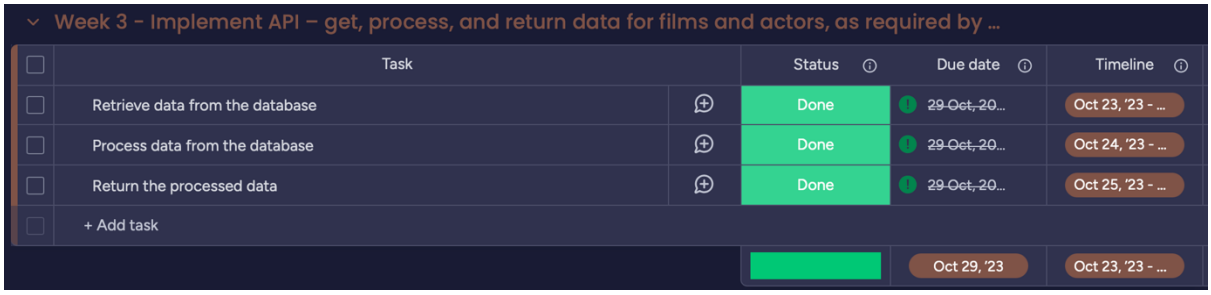


Upcoming Tasks: To implement API – get data, handle data, process and return data as required by other parts of the project.

Week 23rd Oct

Sprint Goal: Implement API – get, process, and return data for films and actors, as required by other parts of the project.

Impediments: We had a bit of an issue with producing the result in well-formed JSON to match the requirements for the groups accessing the data. This was mainly due to the database having multiple tables that needed to be combined when producing the result.



Upcoming Tasks: To further implement API including genres and directors, as well as finalize documentation.

Wek 30th Oct

Sprint Goal: To further implement API including genres and directors, as well as finalize documentation.

Impediments: Some misunderstandings with groups 12 and 14 regarding how to handle filtering data, as we wanted to include certain functionalities in the queries themselves to make them more efficient.



Upcoming Tasks: Outline and plan our game.

Week 6th Nov

Sprint Goal: Game outline and planning – skeleton pages and data displayable.

Impediments: Due to supergroup complications with merging all of the groups' sections we didn't get as much time to focus on thinking about our game aspect.



Upcoming tasks: Start implementing the initial game design.

Week 29th Jan

Sprint goal: Rework API and decide on how to implement the new accessibility requirement.

Impediments: N/A

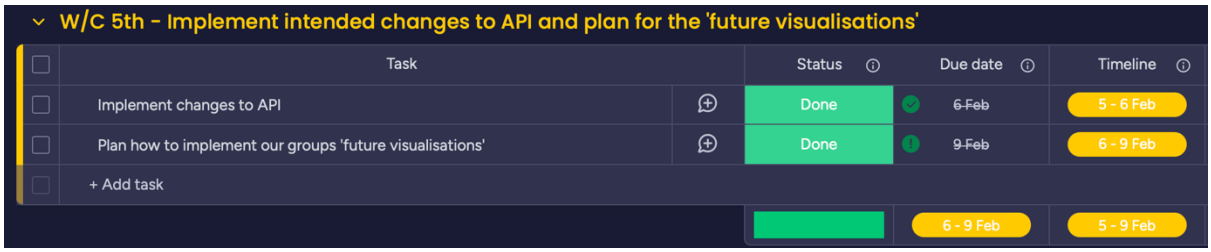


Upcoming tasks: To implement the changes decided upon for the API and plan future visualisations.

Week 5th Feb

Sprint goal: To implement our intended changes to the API and plan our future visualisations.

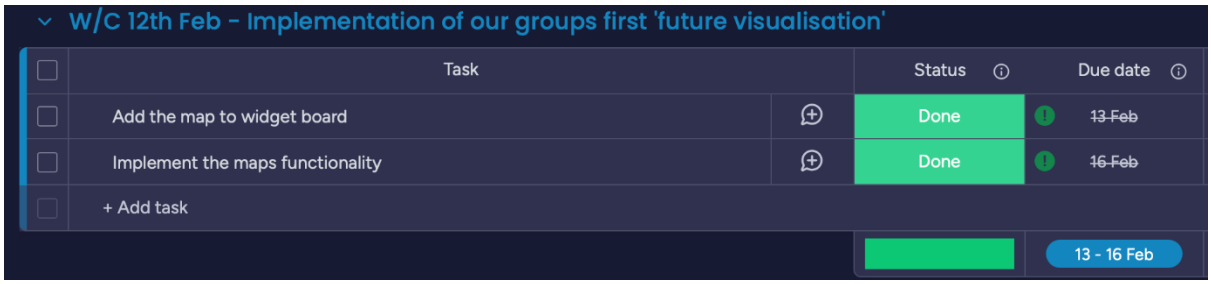
Impediments: When altering the API, we had to communicate with a different group within our supergroup to get these changes completed, this took additional time for the changes to be pushed.



Upcoming tasks: To start implementing the future visualisations that have been planned.

Week 12th Feb

Sprint goal: To implement the first 'future visualisation' of our group. This is the map widget which shows the filming location of the film when inspecting a film in the node map.



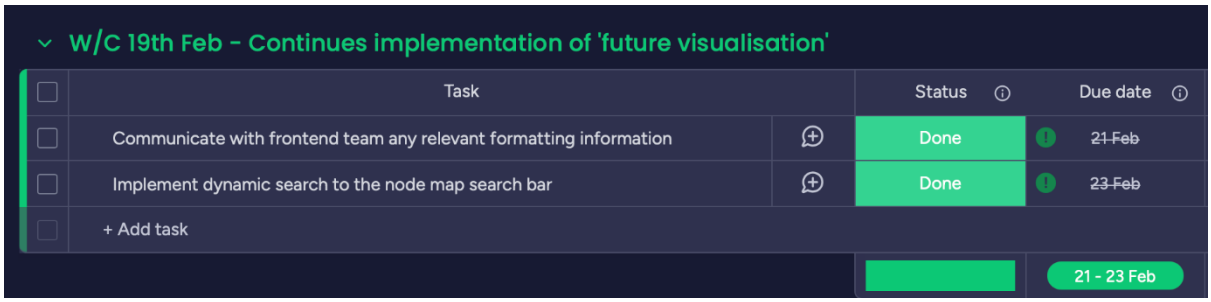
Upcoming tasks: To continue implementing the future visualisation, and some extra added tasks that needed to be completed.

Reflections: Worked well as a team to complete the tasks of this sprint. Gave some extra days to implement the map functionality as from previous sprints it was noted that adding the functionality within the frontend can take extra time.

Week 19th Feb

Sprint goal: To implement the future visualisation, and some extra added tasks that needed to be completed.

Impediments: Communicating with frontend can take extra time but no major problems.



Upcoming tasks: To begin implementation of second 'future' visualisation.

Reflections: That communicating with other groups within supergroup can take extra time, as having to compete with multiple people's schedules. No issues faced, but something to bear in mind for future tasks that require direct communication with supergroup.

Week 4th Mar

Sprint goal: To implement the second 'future visualisation'. This is the interesting facts widgets that are shown for a film

Impediments: N/A

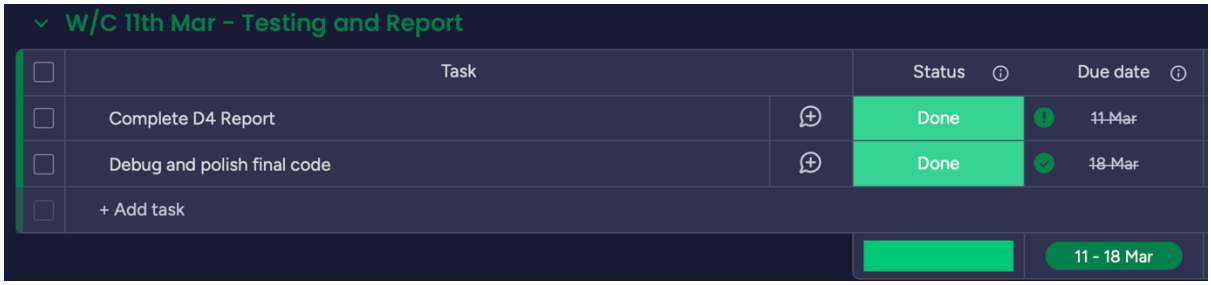


Upcoming tasks: Begin writing the D4 report and debug any issues and polish code.

Reflections: The task of generating queries took slightly longer than expected. This was due to spending extra time thinking of extra queries that could be used. Next time, for a task of a similar nature planning for some extra time will allow for a smoother sprint cycle.

Week 11th Mar

Sprint goal: To write the D4 report and debug any issues and polish off final code.



Upcoming tasks: N/A

Reflections: The debugging and polishing the final code took slightly longer than expected. This was because the task itself is very open ended and doesn't have a clear end point. Next time, maybe being more precise with the description would lead to a more accurate estimated timescale.

Appendix E: API Documentation

The full API documentation is available in PDF format in the **`api-docs`** directory. An interactive version can be viewed at **`http://localhost:3001/docs/`** while the API server is running.

Appendix F: Supergroup Design Document

Supergroup Design Document

Deadline to have project ready for merging and testing: 1 week before deadline
Freeze repository: 3 days before deadline

Scope

This document serves as a standard for each group to work off of when creating new React components and new sections of the web application. It is split into three sections:

UI Standards should contain suggestions for an overall style for the website to adhere to. It should contain specific information about style decisions such as colour palettes, font sizes/families, margins, padding, border size/thickness, etc.

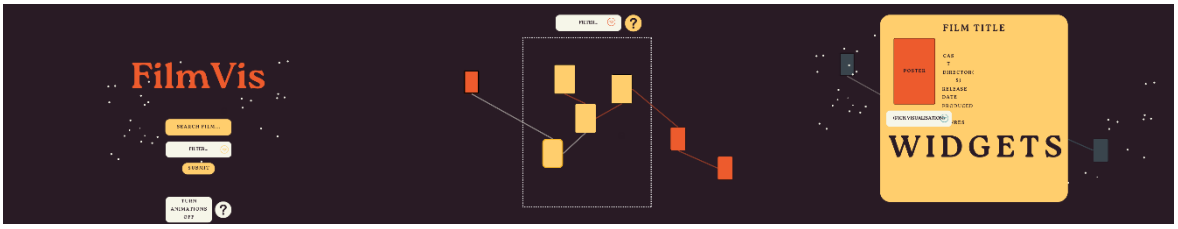
Accessibility Standards should contain suggestions for guidelines to make the content more accessible. This includes but is not limited to high-contrast alternative colour palettes, tab control element ordering, and font size. The ideas in this section can override those specified in the UI standards section if they are included in a separate 'accessibility mode' which the user can choose to toggle to make the web application easier to use for themselves. If they are not part of a separate 'accessibility mode', they should not contradict the specifications in the UI standards section.

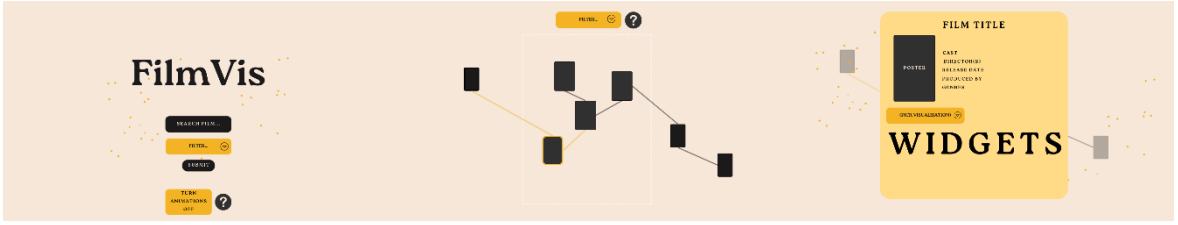
Future Visualisations contains proposals for different visualisations which we have decided to expand out of the movie details screen contained within the node map.

UI Standards

Colour palette generator:
https://venngage.com/tools/accessible-color-palette-generator

WIP website design:
https://www.canva.com/design/DAF6ztnHjLA/ehVBRsFo01_ZURakJQBI7g/edit?utm_content=DAF6ztnHjLA&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton





Accessibility Standards

- Make sure the main content is in the centre of the screen and text is left aligned.
- Clearly label charts. If making a line graph visualisation, for example, use dotted and solid lines as well as colour to differentiate between values.
- Clearly name and label everything.
- Use similarity strengths to dictate order of screen reading and tabbing.
- Screen readers use Aria tags to navigate through webpage. This requires a semantic HTML structure (e.g. <main>, <header> etc.) and alt text for images.
- Have a setting to change fonts to a dyslexia friendly font, OpenDyslexic.
- Have a high contrast mode for colour blind users.
- Designing for the personas we encountered during the Skyscanner workshop, with less focus on blind users.

Screen reader information:
https://medium.com/content-uneditable/lets-create-a-screen-reader-6734fe45aa3d

OpenDyslexic typeface:
https://opendyslexic.org/

Future Visualisations

Widgets for the page that appears when a film in the node map is clicked.

Charts:

https://www.chartjs.org/
https://www.amcharts.com/

Radar chart (Group 12)

- Pie chart showing the distribution of moods selected film fits into, helping users understand the makeup of a film at a glance.
- Voting system for whether the film is actually has those attributes.

Timeline (Group 11)

- Timeline showcasing films with same director, actor, etc.

Film location explorer (Group 13)

- Interactive map widget that allows user to explore the filming location geographically.
- Heat map of number of films with each filming location.

Interesting facts (Group 13)

- Notable statistics about film in sentences.
- Select facts based on attributes that are present and those that are interesting.

Favourites (12)

- Add current film to a stored list of favourites.
- Just for the current session so no need for a new database to store films. Store films locally.

Appendix G: Supergroup Meeting Minutes

Meeting 26/09/23

General

- Website
- Ideas: timeline, map (real-life film locations), genre filtering, seeing regions that produced the most type of movie, map genres to time theme/politics explore history, user profile (building up what a user likes/don't like), gamification, generate a trivia quiz where user sets the constraints, filter by awards
- Story path between movies
- What is visualisation?

Trivia Idea

- Visual where your knowledge lies most
- Pre-set quizzes
- Educate before taking quiz: filter by genres (year, decade), finding overlap between films using SQL queries
- Normalised distribution
- Quiz has data visualisation in the questions.

Visualisation

- User comparison
- Scoreboard/leader boards
- Individual achievements
- User accounts: login system

Group tasks

- Accounts and user data (front end and back)
- Quiz interface (Backend: SQL queries & Frontend: plugins, display)
- Results
- JavaScript

Trivia Backend – Group 11
Trivia Frontend – Group 13
Accounts Backend– Group 14
Accounts Frontend – Group 12

Meeting 28/09/23

Dashboard would be good: multiple features, safety net and versatility

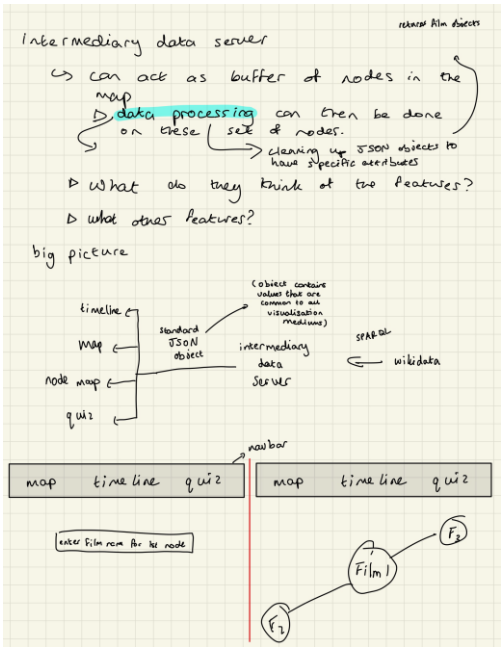
- Quiz
- Map (film locations – with characters to represent genres)
- For some franchises (e.g. Star Wars) have their own visualisation
- Timeline of famous films
- Extra: user scoreboard
- Log in page

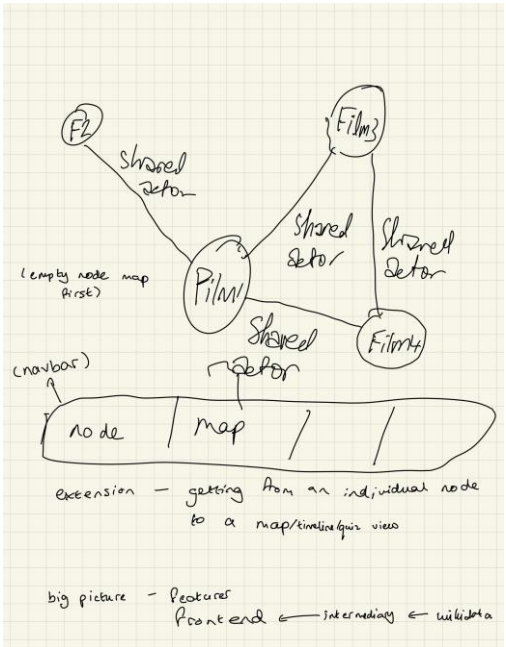
Initial ideas

- Home page: node map (each node is like movies, actors) connected to other movies/actors that are related.
- Filter by genre or decade
- Journey through different tasks (timeline, trivia)
- Trivia for each feature
- Trivia game along the nodes
- Still keep user log in system
- Sharing Wikidata information between different groups
- Users log in to change dashboard to show user's favourite films: create a database for each user and then pass through API to generate for that user

Core idea

- Map of nodes is the starting page
- Each node is an actor/information, connections are co-stars and size of node/line displays strength of connection.
- Integrate timeline (group 11), map (group 12), quiz (group 13)





Meeting 13/10/23

- Determine how the API should communicate with the database and then pass data on.
- JSON objects pulled from Wikidata standardised into something faster to search
- store in a data base, buffer or JSON file
- https://www.wikidata.org/wiki/Wikidata:WikiProject_Movies/Numbers

Front end Wikidata tasks

- Interface
- Search functionality
- List of useful film objects from Wikidata section

Back end Wikidata tasks

- Dealing with storage of JSON objects (e.g. in SQL database or JSON file etc.)

Meeting 23/01/2024

- All groups should contribute code into the supergroup repository in their own branches, merging in as in the proposal sent out earlier
- Make font and colours cohesive together, decide on a colour scheme across the entire application
- Use Google Docs on accessibility design choices
- Consider reducing the number of features, focus on overall quality of project
- Use the node map as the initial interface to move onto more visualisations
- Label issue on GitLab with the group who you need the functionality from
- Keep GitLab layout, rename from groups to what they actually are
- Look into making the application into a multi-page application rather than SPA
- Click on a movie on the node map, get taken to a new page with movie information and visualisations created by groups 11, 12, 13
- Need to make the database API more flexible to accommodate for when we need new fields
- Fetch raw list of movie titles directly from the API, bypass middle server. API to implement searching functionality
- Need to write a design document to adhere to across the entire website
- Spend next week working on accessibility and design documents - ready for January 26, 2024. Everyone adds their own ideas to the documents, and then they are voted on Friday
- Arrow keys to navigate around the node map
- Switch to SCSS - have global CSS with inheritance to make it easier to define with global styles for components
- Look into a better method of hosting: CI/CD, Docker?
- Keep database small for now until we are certain on attributes
- Database/API group coming up with a way to show frontend what is available

Meeting 26/01/2024

General

- Everything ready to merge 2 weeks before the deadline Code freeze 3 days before the deadline
- Add extra key names as needed so that group 11 can repopulate the database
- Add issues to GitLab if new API endpoints are required

Design/Accessibility

- Samina's design generally agreed upon
- Agreed on accessibility mode with dyslexia friendly font and high contrast colour, need to add stuff to align with WCAG guidelines
- Use DOM hierarchy properly
- Use similarity score to determine how to tab through the visualisation
- Milan to look into ARIA tags to make everything more screen reader accessible (also to look into plugins for custom screen readers?)
- Use OpenDyslexic font when in accessibility mode
- Focus on visually impaired as our accessibility target group, starting with high contrast colour and font

Future Visualisations

- Group 12: Radar chart, favourites
- Group 11: Timeline
- Group 13: Map, interesting facts/graphs